

Highlights

Continuity-Aware Crack Segmentation with CCEM and Recall-Sensitive Tversky Training

Author information omitted for review

- A lightweight Crack Continuity Enhancement Module is introduced for crack segmentation.
- CCEM enhances elongated crack structures using local, strip, and dilated branches.
- A recall-sensitive Tversky loss mitigates sparse-pixel and thin-crack missed detection.
- Final results are reported with ODS/OIS threshold sweeping instead of fixed 0.5 threshold.
- The final method improves average ODS metrics with only a small complexity increase.

Continuity-Aware Crack Segmentation with CCEM and Recall-Sensitive Tversky Training

Author information omitted for review^a

^a*Institution information omitted for review,*

Abstract

Pixel-level crack segmentation is challenging because crack regions are sparse, thin, elongated, and easily confused with background texture. This paper presents a lightweight continuity-aware improvement for MixerCseg. The final network follows the pipeline TransMixer, DEGConv, CCEM, SRF, and SegHead, where the proposed Crack Continuity Enhancement Module (CCEM) is inserted after DEGConv and before SRF at all stages. CCEM combines local, strip, and dilated depthwise branches with gated residual fusion to enhance crack continuity and structural representation while suppressing background false positives. In addition, a recall-sensitive Tversky loss is added to the original BCE and Dice objective to reduce missed detections of sparse crack pixels. The final training objective is BCE plus 5 times Dice plus 0.2 times Tversky with $\alpha = 0.3$ and $\beta = 0.7$. All final results are recomputed using a standard threshold-sweeping evaluation protocol rather than training-log precision and recall. On CrackMap, DeepCrack, and CamCrack789, the final method achieves average mIoU@ODS, ODS-F1, OIS-F1, precision, and recall of 0.8670, 0.8503, 0.8479, 0.8411, and 0.8606, respectively. Compared with the baseline, the average gains are 0.0062 mIoU@ODS, 0.0083 ODS-F1, 0.0081 OIS-F1, and 0.0169 precision, with recall remaining stable. The final method only increases parameters from 2.542M to 2.597M and FLOPs from 1.989G to 2.059G, showing a favorable accuracy-efficiency trade-off.

Keywords: Crack segmentation, Continuity enhancement, Tversky loss, Threshold sweeping, Lightweight segmentation, Structural defect detection

1. Introduction

Crack segmentation is an important task in pavement, bridge, and concrete structure inspection. Accurate pixel-level crack maps support quantitative condition assessment, maintenance planning, and automated visual inspection. However, crack segmentation remains difficult because crack pixels occupy only a small portion of an image, often appear as thin and discontinuous structures, and can be confused with shadows, stains, joints, aggregates, or other background texture.

Deep convolutional segmentation networks have substantially improved crack detection accuracy compared with hand-crafted feature methods. Encoder-decoder networks such as FCN and U-Net provide dense predictions, while edge-aware models and transformer-based architectures further improve context modeling. Nevertheless, many segmentation models still face a precision-recall trade-off in crack scenes. Strong context modeling can suppress false positives but may break thin crack continuity, whereas recall-oriented training can recover weak cracks but may introduce background noise. For practical inspection, both continuous crack structures and controlled false positives are needed.

This work improves the existing MixerCSeg crack segmentation framework with minimal architectural changes. The final network structure is fixed as

$$\text{TransMixer} \rightarrow \text{DEGConv} \rightarrow \text{CCEM}(\text{full}) \rightarrow \text{SRF} \rightarrow \text{SegHead}.$$

The proposed Crack Continuity Enhancement Module (CCEM) is placed after DEGConv and before SRF and is applied to all stages. CCEM is designed to strengthen local crack detail, elongated directional continuity, and small-gap connection using local, strip, and dilated depthwise branches. A gated residual fusion mechanism modulates the enhanced features before adding them back to the original representation.

The training objective is also adjusted in a low-risk manner. The original BCE plus Dice objective is extended with a Tversky term:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + 5\mathcal{L}_{\text{Dice}} + 0.2\mathcal{L}_{\text{Tversky}},$$

where $\alpha = 0.3$ and $\beta = 0.7$. The Tversky term is used only during training and therefore does not change inference complexity. To avoid misleading conclusions caused by a fixed or implicit threshold, all main results are reported using a standard ODS/OIS threshold-sweeping protocol.

The main contributions are summarized as follows:

- A lightweight CCEM is introduced to improve crack continuity and structural representation in MixerCSeg without changing the backbone or data pipeline.
- A recall-sensitive Tversky training strategy is used to reduce missed detections of sparse and thin crack pixels.
- A standard evaluation protocol is adopted, where ODS/OIS threshold sweeping is used as the main reporting criterion and fixed threshold 0.5 is retained only as reference.
- Experiments on CrackMap, DeepCrack, and CamCrack789 show consistent average improvements in mIoU@ODS, ODS-F1, OIS-F1, and precision with a small increase in model complexity.

2. Related Work

2.1. Crack Segmentation

Early crack detection methods relied on low-level image cues, thresholding, morphology, and random forest classifiers. These approaches are sensitive to illumination changes and complex background texture. Deep learning methods, especially fully convolutional networks, have become the dominant choice for dense crack prediction. DeepCrack and related hierarchical feature learning models demonstrated the effectiveness of multi-scale convolutional features for crack detection. Encoder-decoder models further improved dense prediction by combining semantic and spatial information.

Despite these advances, crack segmentation still suffers from thin-structure discontinuity and background confusion. Crack pixels are highly imbalanced against background pixels, and their appearance varies across datasets. Therefore, modules that explicitly enhance crack continuity and training objectives that reduce false negatives are useful for improving robustness.

2.2. Dense Prediction Networks

FCN first formulated semantic segmentation as end-to-end dense prediction. U-Net introduced a symmetric encoder-decoder architecture with skip connections and became a standard baseline for pixel-level segmentation.

Later models such as DeepLab and SegFormer improved contextual modeling using atrous convolution and transformer-based encoders. For crack segmentation, however, large context modules may not always improve recall, because thin cracks can be suppressed if their local structure is not preserved. This motivates a lightweight continuity-aware module tailored to elongated crack patterns.

2.3. Loss Functions for Imbalanced Segmentation

Class imbalance is a common problem in medical and defect segmentation. Dice loss directly optimizes overlap and is widely used together with BCE. Tversky loss generalizes Dice by assigning different penalties to false positives and false negatives. By setting a larger false-negative penalty, Tversky loss becomes suitable for recall-sensitive segmentation. In this work, Tversky loss is used only as a training strategy and does not change the inference architecture.

3. Method

3.1. Overall Architecture

The final architecture keeps the original MixerCSeg pipeline and only inserts CCEM after DEGConv and before SRF. Given an input image, the stem and TransMixer encoder extract multi-stage features. DEGConv processes these features, CCEM enhances crack continuity at all stages, SRF fuses multi-level representations, and the segmentation head predicts the final crack probability map. No BRM, MSCM, SGH, or other additional modules are retained in the final model.

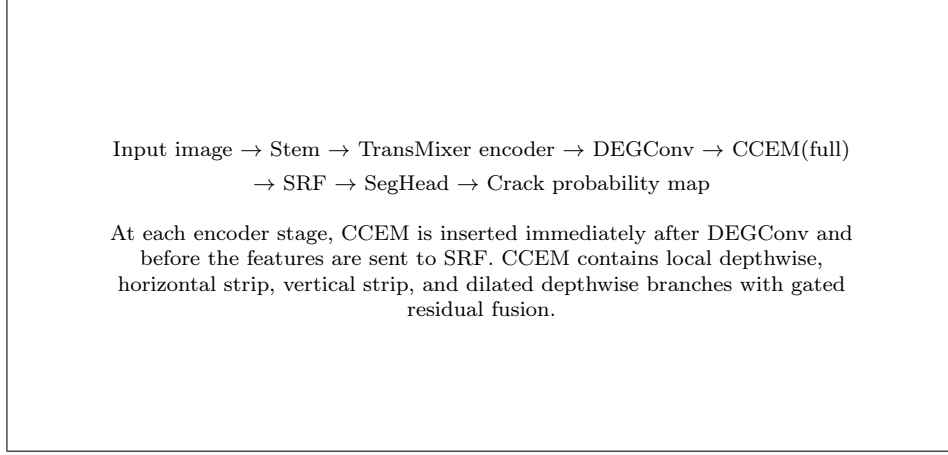


Figure 1: Overall architecture of the final method. CCEM is inserted after DEGConv at all encoder stages while the original MixerCseg data pipeline and decoder are kept unchanged.

3.2. Crack Continuity Enhancement Module

Let $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$ denote an input feature map after DEGConv. CCEM first applies a 1×1 projection to reduce channels:

$$\mathbf{F}_r = \phi_{1 \times 1}(\mathbf{F}), \quad (1)$$

where ϕ denotes convolution followed by normalization and activation.

The reduced feature is then sent to three continuity-aware branches:

$$\mathbf{B}_{local} = \phi_{3 \times 3}^{dw}(\mathbf{F}_r), \quad (2)$$

$$\mathbf{B}_{strip,h} = \phi_{1 \times k}^{dw}(\mathbf{F}_r), \quad (3)$$

$$\mathbf{B}_{strip,v} = \phi_{k \times 1}^{dw}(\mathbf{F}_r), \quad (4)$$

$$\mathbf{B}_{dilated} = \phi_{3 \times 3, d}^{dw}(\mathbf{F}_r), \quad (5)$$

where dw indicates depthwise convolution and d is the dilation rate. The local branch captures fine crack details, the strip branches model horizontal and vertical elongated structures, and the dilated branch helps bridge small discontinuities.

The branch outputs are concatenated and fused by a 1×1 convolution:

$$\mathbf{F}_{fuse} = \phi_{1 \times 1}([\mathbf{B}_{local}, \mathbf{B}_{strip,h}, \mathbf{B}_{strip,v}, \mathbf{B}_{dilated}]). \quad (6)$$

Then a gate is used to modulate the enhanced feature:

$$\mathbf{G} = \sigma(\psi_{1 \times 1}(\mathbf{F}_{fuse})), \quad \mathbf{F}_g = \mathbf{G} \odot \mathbf{F}_{fuse}. \quad (7)$$

Finally, CCEM uses residual fusion:

$$\mathbf{F}_{out} = \mathbf{F} + \gamma \mathbf{F}_g, \quad (8)$$

where γ is a learnable scalar initialized to zero. This residual design stabilizes training and avoids destroying the original features at the beginning of optimization.

3.3. Training Loss

The final training loss combines BCE, Dice, and Tversky:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + 5\mathcal{L}_{\text{Dice}} + 0.2\mathcal{L}_{\text{Tversky}}. \quad (9)$$

Dice loss is computed from the predicted probability map and the binary ground truth. Tversky loss is defined as

$$\mathcal{L}_{\text{Tversky}} = 1 - \frac{TP + \epsilon}{TP + \alpha FP + \beta FN + \epsilon}, \quad (10)$$

where TP , FP , and FN denote true positives, false positives, and false negatives, respectively. We set $\alpha = 0.3$ and $\beta = 0.7$, assigning a larger penalty to missed crack pixels. Since the Tversky term is only used during training, the inference complexity of the final method is identical to CCEM full.

4. Experimental Setup

4.1. Datasets

Experiments are conducted on three crack segmentation datasets: CrackMap, DeepCrack, and CamCrack789. The same data loading and preprocessing pipeline as the baseline implementation is used. No dataloader or dataset split is changed when adding CCEM or Tversky training.

Dataset statistics

Dataset	Train/Val/Test	Preprocessing	Description	Source split
CrackMap	84/12/24	resized to 512×512	Road crack segmentation dataset	project split
DeepCrack	368/53/106	resized to 512×512	Crack segmentation dataset with diverse scenes	project split
CamCrack789	553/79/157	resized to 512×512	Structural surface crack segmentation dataset	project split

The listed splits are the data partitions used in the current project directory. All variants use the same split and resizing pipeline.

4.2. Compared Methods

Three methods are compared:

- **Baseline:** the original MixerCSeg model without CCEM.
- **CCEM full:** the baseline model with CCEM inserted after DEGConv and before SRF at all stages.
- **CCEM full + Tversky:** the final method, using CCEM full and the BCE plus Dice plus Tversky training loss.

4.3. Evaluation Protocol

All final results are recomputed from saved prediction and label images using a standard offline evaluation script. The saved predictions are grayscale response maps generated by the inference pipeline, and the same threshold-sweeping procedure is applied to every compared method. The main metrics are ODS threshold, mIoU@ODS, ODS-F1, OIS-F1, precision@ODS, and recall@ODS. ODS is obtained by sweeping thresholds from 0.00 to 0.99 and selecting the dataset-level threshold with the best F1. OIS is computed by selecting the best F1 threshold for each image and then averaging over images.

The fixed threshold 0.5 results are reported only as reference. They are not used as the main conclusion because different methods can have different probability calibration. Training-log F1, precision, and recall are also not used as final paper results.

5. Results and Discussion

5.1. Main Quantitative Results

Table 1 reports the standard ODS/OIS results on the three datasets. The final method achieves the best CrackMap and CamCrack789 results and remains competitive on DeepCrack. Compared with the baseline, the final method improves the three-dataset average mIoU@ODS from 0.8608 to 0.8670 and ODS-F1 from 0.8420 to 0.8503.

Table 1: Standard evaluation results using ODS/OIS threshold sweeping.

Dataset	Method	ODS thr	mIoU@ODS	ODS-F1	OIS-F1	P@ODS	R@ODS
CrackMap	Baseline	0.30	0.8114	0.7786	0.7781	0.7329	0.8304
CrackMap	CCEM full	0.00	0.8142	0.7824	0.7892	0.7463	0.8223
CrackMap	CCEM full + Tversky	0.10	0.8228	0.7943	0.8007	0.7616	0.8299
DeepCrack	Baseline	0.09	0.9259	0.9234	0.9181	0.9210	0.9258
DeepCrack	CCEM full	0.00	0.9273	0.9250	0.9146	0.9262	0.9238
DeepCrack	CCEM full + Tversky	0.00	0.9272	0.9249	0.9170	0.9207	0.9290
CamCrack789	Baseline	0.01	0.8450	0.8240	0.8234	0.8188	0.8293
CamCrack789	CCEM full	0.00	0.8470	0.8266	0.8227	0.8295	0.8238
CamCrack789	CCEM full + Tversky	0.00	0.8510	0.8317	0.8261	0.8409	0.8227

The averaged results are shown in Table 2. CCEM full mainly improves precision, indicating that the continuity-enhancement branches help suppress background false positives. Adding the Tversky term further improves the averaged mIoU@ODS, ODS-F1, and OIS-F1 while keeping recall nearly unchanged compared with the baseline.

Table 2: Three-dataset average results under the ODS protocol.

Method	mIoU@ODS	ODS-F1	OIS-F1	P@ODS	R@ODS
Baseline	0.8608	0.8420	0.8399	0.8242	0.8618
CCEM full	0.8629	0.8447	0.8422	0.8340	0.8566
CCEM full + Tversky	0.8670	0.8503	0.8479	0.8411	0.8606

5.2. Qualitative Results

Figures 2–4 show visual comparisons on CrackMap, DeepCrack, and CamCrack789. Each prediction mask is binarized using the corresponding method’s own ODS threshold rather than a fixed 0.5 threshold. This visualization protocol is consistent with the quantitative main table.

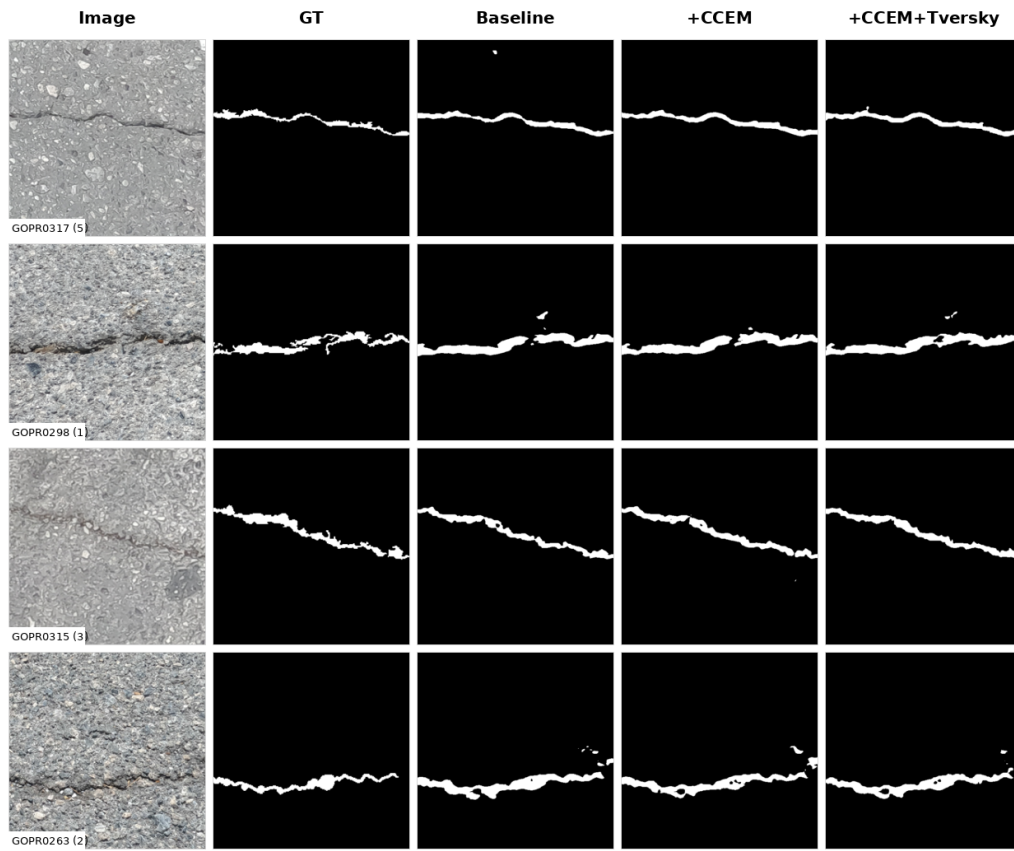


Figure 2: Qualitative comparison on CrackMap. Predictions are binarized using each method's ODS threshold.

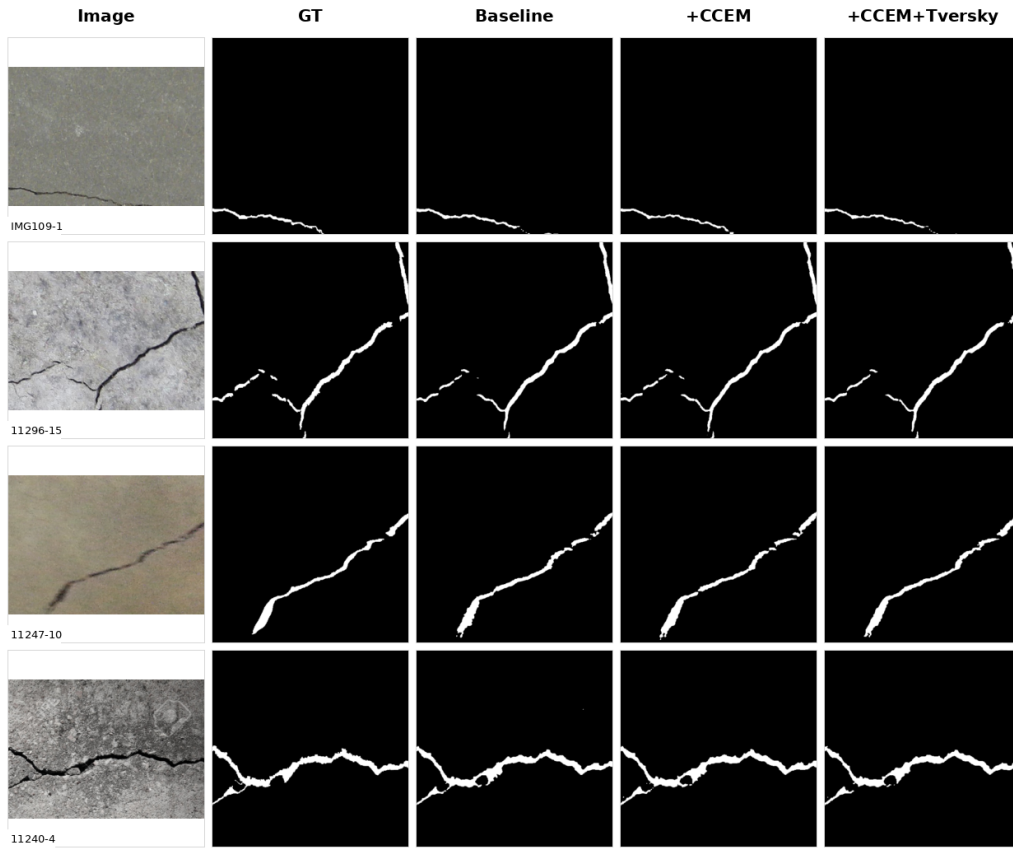


Figure 3: Qualitative comparison on DeepCrack. Predictions are binarized using each method’s ODS threshold.

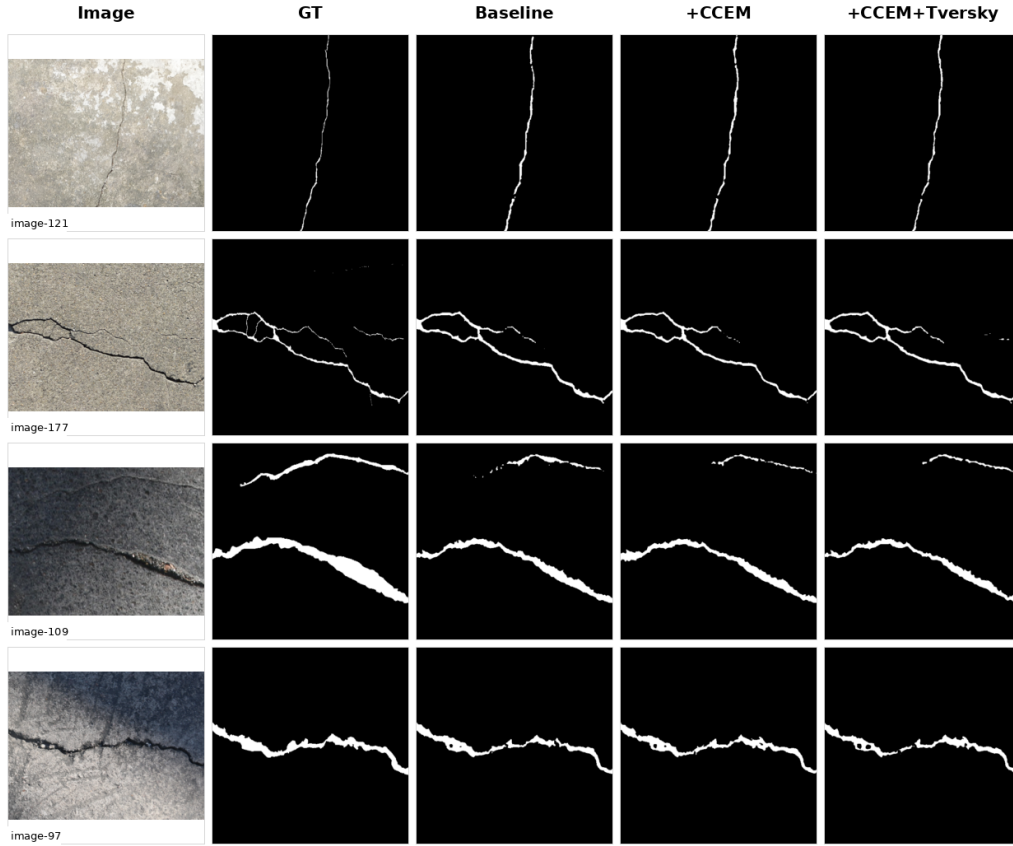


Figure 4: Qualitative comparison on CamCrack789. Predictions are binarized using each method’s ODS threshold.

6. Complexity Analysis

Table 3 compares the computational complexity of the baseline and the final method. Because Tversky loss is only used during training, the inference complexity of CCEM full plus Tversky is identical to CCEM full. The final method increases parameters by only 0.055M and FLOPs by 0.070G. The latency increases from 165.088 ms per image to 167.684 ms per image, indicating that the accuracy improvement is obtained with a small computational overhead. The model size increases from 9.715 MB to 9.926 MB, and the peak GPU memory changes slightly from 195.798 MB to 196.034 MB.

Table 3: Complexity comparison. The final method has the same inference complexity as CCEM full because Tversky loss is used only during training.

Model	Params(M)	FLOPs(G)	FPS	Latency(ms/img)	Δ Params	Δ FLOPs
Baseline	2.542	1.989	6.057	165.088	0.000	0.000
CCEM full / final method	2.597	2.059	5.964	167.684	0.055	0.070

7. Ablation Study

The final method was selected after comparing several candidate extensions under the standard ODS/OIS protocol. On CrackMap, CCEM full improved the standard ODS-F1 from 0.7786 to 0.7824, and the final CCEM full plus Tversky setting further improved mIoU@ODS to 0.8228 and ODS-F1 to 0.7943. Additional modules, including BRM, MSCM, and SGH, did not outperform CCEM full and were therefore not retained. Low-risk CCEM variants such as LeakyGate and BranchWeight also failed to exceed the current CrackMap best result of mIoU@ODS 0.8228 and ODS-F1 0.7943. Therefore, the final architecture keeps only CCEM full, and the final training strategy adds the Tversky term.

The ablation trend suggests that the main structural benefit comes from continuity-aware feature enhancement rather than stacking additional modules. The Tversky term complements CCEM by improving recall sensitivity during training while leaving inference unchanged.

8. Conclusion

This paper presents a lightweight continuity-aware improvement for crack segmentation. The final network inserts CCEM into MixerCseg after DEG-Conv and before SRF at all stages. CCEM enhances local detail, elongated directional structures, and small crack gaps through local, strip, and dilated branches with gated residual fusion. A recall-sensitive Tversky loss is further added to the BCE and Dice training objective to reduce missed crack pixels. Standard evaluation on CrackMap, DeepCrack, and CamCrack789 shows that the final method improves average mIoU@ODS, ODS-F1, OIS-F1, and precision while keeping recall stable. The added computational cost is small, making the method suitable for efficient crack segmentation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and Code Availability

The experiments use public crack segmentation benchmarks organized under the project data protocol. The implementation, evaluation scripts, result summaries, and visualization materials are maintained in the project repository.

References

- [1] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3431–3440.
- [2] O. Ronneberger, P. Fischer, T. Brox, U-Net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention, 2015, pp. 234–241.
- [3] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam, Encoder-decoder with atrous separable convolution for semantic image segmentation, in: Proceedings of the European Conference on Computer Vision, 2018, pp. 801–818.
- [4] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, P. Luo, SegFormer: Simple and efficient design for semantic segmentation with transformers, in: Advances in Neural Information Processing Systems, 2021, pp. 12077–12090.
- [5] Y. Shi, L. Cui, Z. Qi, F. Meng, Z. Chen, Automatic road crack detection using random structured forests, IEEE Transactions on Intelligent Transportation Systems 17 (12) (2016) 3434–3445.
- [6] Q. Zou, Z. Zhang, Q. Li, X. Qi, Q. Wang, S. Wang, DeepCrack: Learning hierarchical convolutional features for crack detection, IEEE Transactions on Image Processing 28 (3) (2019) 1498–1512.

- [7] Y. Liu, J. Yao, X. Lu, R. Xie, L. Li, DeepCrack: A deep hierarchical feature learning architecture for crack segmentation, *Neurocomputing* 338 (2019) 139–153.
- [8] S. Xie, Z. Tu, Holistically-nested edge detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 1395–1403.
- [9] F. Milletari, N. Navab, S.-A. Ahmadi, V-Net: Fully convolutional neural networks for volumetric medical image segmentation, in: *2016 Fourth International Conference on 3D Vision*, 2016, pp. 565–571.
- [10] S. S. M. Salehi, D. Erdogmus, A. Gholipour, Tversky loss function for image segmentation using 3D fully convolutional deep networks, in: *International Workshop on Machine Learning in Medical Imaging*, 2017, pp. 379–387.
- [11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, in: *International Conference on Learning Representations*, 2021.
- [12] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, Swin Transformer: Hierarchical vision transformer using shifted windows, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10012–10022.
- [13] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, *arXiv preprint arXiv:2312.00752*.
- [14] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, Y. Liu, VMamba: Visual state space model, *Advances in Neural Information Processing Systems* 37 (2024) 103031–103063.
- [15] Z. Zhao, Z. Ding, P. Niu, W. Sun, F. Guo, MixerCSeg: An efficient mixer architecture for crack segmentation via decoupled Mamba attention, *arXiv preprint arXiv:2603.01361* (2026).
- [16] H. Liu, C. Jia, F. Shi, X. Cheng, S. Chen, SCSegamba: Lightweight structure-aware vision Mamba for crack segmentation in structures, in:

Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 29406–29416.

- [17] Y. Gu, Y. Meng, K. Zheng, X. Sun, J. Ji, W. Ruan, L. Cao, R. Ji, An efficient and mixed heterogeneous model for image restoration, arXiv preprint arXiv:2504.10967 (2025).